



FACULTY OF NATURAL, MATHEMATICAL, AND
ENGINEERING SCIENCES
DEPARTMENT OF INFORMATICS

Software Engineering Group Project

Student Union Club Management Platform

miHoyo
Mingze Li
Ruijie Li
Qilan Lin
Peize Li
Jiale She
Yu-Tang Huang
Youyou Wu

March 27, 2025

Contents

1	Introduction	1
2	Objectives and stakeholders	2
2.1	Project objectives	2
2.2	Stakeholder	2
2.2.1	Stakeholder analysis	2
2.2.2	Key stakeholders	3
3	Specifications	4
3.1	Functional specifications	4
3.2	Non-functional specifications	7
4	Project management	9
5	Design and implementation	11
5.1	System Architecture	11
5.2	Technology Stack Selection	12
5.2.1	Backend Technologies	12
5.3	Alternative Design Options and Justifications	12
5.3.1	Full CMS Solutions vs. Custom-coded Implementation	12
5.3.2	Other Web Frameworks	12
5.3.3	Monolithic vs. Microservices Architecture	12
5.3.4	Frontend Technologies	13
5.3.5	Storage Solutions	13
5.4	Data Model Design	13
5.4.1	User Model	13
5.4.2	Event Model	13
5.4.3	RSVP Model	13
5.5	Security Mechanism Implementation	13
5.5.1	Authentication and Authorization	13
5.5.2	CSRF Protection	14
5.5.3	Password Policy	14
5.6	User Interface Design	14
5.6.1	Design Style and Principles	14
5.6.2	Visual Design System	14
5.6.3	Component Design	15
5.6.4	Responsive Design Implementation	15
5.6.5	Interaction Design Patterns	15

5.7	Performance Optimization	16
5.7.1	Static Resource Handling	16
5.7.2	Caching Strategy	16
5.7.3	Lazy Loading	16
5.8	Extensibility Design	16
5.8.1	Modular Plugin System	16
5.8.2	API Interface Design	16
5.8.3	Internationalization Support	16
5.9	Challenges and Solutions	16
5.9.1	Event Management System	16
5.9.2	Dynamic Form Handling	17
5.9.3	Notification System Integration	17
5.10	Testing Strategy	17
5.11	Conclusion	17
6	Testing	18
6.1	Approach and tools	18
6.2	Quality assurance processes	18
6.3	Evaluation of testing	18
A	Manual tests	20

Chapter 1

Introduction

Business objectives

This project aims to provide student societies with a secure, privacy-focused alternative to mainstream social media platforms for managing their online presence. By creating a dedicated web application, societies can centralize membership tracking, news sharing, and event coordination while avoiding the data exploitation and polarization prevalent on third-party platforms. Universities and student unions gain oversight capabilities to approve, monitor, or revoke society pages, ensuring alignment with institutional guidelines. The platform prioritizes branding flexibility, allowing societies to tailor their pages to reflect their unique identities and organizational needs, fostering inclusivity for diverse groups ranging from academic clubs to competitive teams.

System Statement

Built as a scalable Django web application, the platform operates across desktop and mobile browsers, utilizing Python for backend logic with Django's ORM managing SQLite databases. The frontend combines HTML/CSS templates with React-based interactive components, enhanced by Django-summernote's rich text editing for content management systems. Modular architecture enables:

- Dynamic widget customization through React Grid Layout
- RESTful API integrations via Axios
- Progressive enhancement with Vite-built frontend modules

Authentication and permissions leverage Django's built-in tools, while container-ready deployment supports both on-premises and cloud infrastructure.

Chapter 2

Objectives and stakeholders

2.1 Project objectives

The primary objective of this project is to create a customized online platform for students, allowing them to freely create or join clubs and participate in club activities. The platform will provide an alternative to mainstream social media platforms, enabling student organizations to control their data, branding, and communications without the risk of data exploitation or unnecessary commercialization.

The system aims to add value for various stakeholders in the following ways: Clubs can track memberships, communicate with members, and efficiently organize events. Clubs can create news, updates, and media in a structured manner. Website administrators and club administrators monitor and approve club pages to ensure compliance with institutional policies. Clubs can design pages that reflect their identity and cater to the needs of diverse groups, ranging from academic clubs to sports teams.

By achieving these objectives, the platform enhances student community engagement while maintaining privacy and institutional oversight.

2.2 Stakeholder

The platform serves multiple stakeholder groups, including students, general users, club managers, club members, and website administrators.

Stakeholders can be categorized into primary and secondary groups based on their level of involvement and influence.

2.2.1 Stakeholder analysis

Primary Stakeholders

- **Student Clubs:**
 - The main users of the platform, responsible for creating and managing their club pages, events, and communications.
- **Club Managers:**

- Responsible for approving, monitoring, and managing club pages.
- **General Users:**
 - Can browse the website and club pages, and after registration, can apply to join clubs.
- **Club Members:**
 - End-users who join clubs, participate in events, and engage with content.

Secondary Stakeholders:

- **Developers and System Administrators:**
 - Responsible for maintaining and updating the platform, ensuring security and scalability.
- **Event Sponsors and Partners:**
 - External organizations that may support student clubs through sponsorships or event collaborations.
- **Alumni and External Members:**
 - Former students who wish to stay connected with clubs and participate in activities.

2.2.2 Key stakeholders

Among all stakeholders, the following groups are critical to the platform’s success:

- **Student Clubs:**
 - As the primary users, their engagement and satisfaction determine the platform’s adoption and effectiveness.
- **Developers and System Administrators:**
 - They ensure the system remains functional, secure, and up-to-date.

By addressing the needs and concerns of these key stakeholders, the project aims to establish a sustainable and widely adopted system for student clubs.

Chapter 3

Specifications

3.1 Functional specifications

1. User Registration and Authentication System

- Feature Description:
 - Allows users to register new accounts using email and log in with a username and password. Includes password reset. Built with Django's built-in authentication module for secure access.
- How to Access:
 - Homepage avatars provide access to the registration and login pages when not logged in, and to the Dashboard for account settings once logged in.
- Role Access:
 - All users must log in to access society club features. Admin features require admin role.
- Business Goal:
 - Ensures user data privacy and platform control by avoiding third-party login dependencies.

2. Customizable Club Hub

- Feature Description:
 - Enables club members to personalize club homepages, share their ideas with image uploads, notices, event widgets, and clocks using React Grid Layout.
- How to Access:
 - Click "Club hubs" on the Navbar then select club.
- Role Access:
 - Every club members can access, view and edit.
- Business Goal:

- Helps clubs build their identity and promote diversity on the platform.

3. Member Management

- Feature Description:
 - Club Manager can add/remove members and assign roles
- How to Access:
 - Club Manager access club detail page and click "Club Manager Page" can find membership management function
- Role Access:
 - Only club manager
- Business Goal:
 - Enhances structural efficiency and supports autonomous society club operations.

4. News Publishing

- Feature Description:
 - Users can publish news using a rich text editor (Django Summernote).
- How to Access:
 - Click "News" in the Navbar, then "Publish News"
- Role Access:
 - Regular users can only post news in their own name, Club Manager can post news in the name of the club.
- Business Goal:
 - Improves internal communication and reduces reliance on external platforms.

5. Forum Post and Reply System

- Feature Description:
 - Users can create and reply to forum posts to facilitate discussion.
- How to Access:
 - Click on "Forums" in the Navbar to access related functions.
- Role-Based Access:
 - Logged-in users can post and reply in their own name, Club Manager can post forum in the name of the club.
- Business Goal:
 - Promotes communication and knowledge sharing.

6. Multi-type Notification Management

- Feature Description:
 - Supports various notification types like requests status, membership update.
- How to Access:
 - Notification menu(little bell) in the top navigation bar.
- Role-Based Access:
 - All logged-in users can receive relevant notifications.
- Business Goal:
 - Enhances timely and relevant communication.

7. Event Registration System

- Feature Description:
 - User can RSVP events, check event status, add event into their calendar and cancel if needed.
- How to Access:
 - User can access by click "Events" in Navbar
- Role-Based Access:
 - All regular users can access and RSVP events.
 - Club manager can create and modify events from "Club manager page - events"
- Business Goal:
 - Ensures orderly and justified creation of new societies.

8. New Club Request and Approval

- Feature Description:
 - Users can request to create new clubs. Admins can approve or reject applications.
- How to Access:
 - "Create Club" button on the bottom of club list page, "admin panel - requests" for review.
- Role Access:
 - Users can apply. Admins can approve/reject.
- Business Goal:
 - Clarifies structure and responsibilities within clubs.

9. System Administrator Panel

- Feature Description:

- The System Administrator can review New Club Requests, modify or delete content (Memberships, News, Events, Clubs, etc.) in the system to ensure compliance with policies.
- How to Access:
 - After System Administrator login, click "Admin Panel" in Navbar.
- Role Access:
 - System Administrator Only
- Business Goal:
 - Supports institutional governance and ensures content compliance.

10. Search and Filtering Capability

- Feature Description:
 - Most lists within the system provide search and filtering functionalities, including:
 - * Exploratory lists accessible to general users, such as Clubs, Events, News, and Forums.
 - * Management-oriented lists available to Club Managers and Administrators, such as Memberships and Requests.
- How to Access:
 - Search and filtering tools are positioned at the top of most tables.
 - For example, in the Event List:
 - * After navigating to "Events" via the Navbar, users can find the search and filtering tools located above the event listings.
- Role-Based Access:
 - All users have access to search and filter functionalities, although specific searchable content varies based on user permissions.
- Business Goal:
 - Enhance user experience and lower the entry barrier for users to easily find relevant information.

3.2 Non-functional specifications

1. Security

- Feature Description:
 - Ensure the security, confidentiality, integrity and availability of user and system data by preventing unauthorized access.
- Specific measures:

- Passwords are securely stored using Django’s built-in authentication.
- CSRF protection is enabled via `CSRF_TRUSTED_ORIGINS`.
- Using the mixin method in key apps enables code reuse, centralized management of permissions, modular security checks, and enhances the flexibility and maintainability of the view layer through combinatorial inheritance. For example:
 - * `UserTypeRequiredMixin` implements user type whitelisting control via the `allowed_types` parameter
 - * `LoginRequiredMixin` Harmonizes the handling of non-logged-in user redirects.
 - * `ClubManagerRequiredMixin` wraps the club manager permission checking logic

2. Usability

- Feature Description:
 - The platform is intuitive, easy to learn and use, with a clear interface aligned with user expectations.
- Specific measures:
 - Cross-application style harmonization through global CSS variables and component libraries (e.g. Bootstrap)
 - Three-tier information structure (Navbar → Left menu → Card Controls) for Club Manager Page, User Dashboard and Admin Panel to reduce the cognitive load on users.
 - Implemented search function in most of the forms to reduce the user's difficulty. For example:
 - * Search box at the top of the "Clubs" page

3. Extensibility

- Feature Description:
 - The system architecture supports seamless addition of new features and modules.
- Specific measures:
 - Each sub-app maintains its own routing configuration, which is dynamically integrated through the project's root URLs file, making it easy to extend at a later stage.
 - The left menu bar of the Club Manager Page, User Dashboard and Admin Panel is modular and inheritable, allowing for the expansion of new control functions.
 - Club Hub uses the Widget design pattern, which can be extended to other application scenarios at a later stage.

Chapter 4

Project management

Throughout the project, our team adopted a hybrid approach to project management, combining traditional planning with Agile-inspired iterative development. This strategy enabled us to maintain organization while remaining responsive to evolving requirements and challenges.

Planning and Tooling

Initially, we used **Trello** to outline key deliverables and divide the project into clear, actionable tasks. Each task was clearly assigned and categorized into columns such as "To Do," "In Progress," and "Completed," enhancing transparency and accountability among team members.

Version control and collaboration were handled through **GitHub**, where each team member created a new branch every week. This approach allowed us to clearly manage version control by individual contribution rather than by feature, making it easier to pinpoint issues and revert changes if necessary. Pull requests were regularly reviewed and approved by teammates before merging into our main development branch, ensuring code quality and consistency.

Iterative Progress and Agile Practices

While we initially created a comprehensive roadmap, we consistently applied Agile practices, holding weekly meetings to review progress and adjust our priorities accordingly. For example, three weeks before the final deadline, we discovered our goal of creating a wide variety of customizable components in the Club Hub was overly ambitious given our timeline. Consequently, we decided to simplify this feature, implementing only essential components such as images, clocks, and text. However, we strategically designed our codebase to facilitate future expansions.

Another example occurred during the middle of our development when we identified that implementing a user deletion feature was highly problematic due to extensive foreign key dependencies in the database. Recognizing the risk to data integrity, we proactively decided to abandon this feature. Such flexibility significantly reduced potential issues and maintained project stability.

Communication and Collaboration

Our communication extended beyond scheduled meetings, as team members frequently interacted privately through messaging platforms to address tightly coupled features. For instance, frontend and backend developers regularly coordinated to ensure seamless integration of the event management functionalities, which required continuous adjustments to the Django views and frontend templates. This constant dialogue was crucial in resolving unexpected issues promptly and maintaining steady progress.

Evaluation and Lessons Learned

In this project, we effectively utilized version control tools to ensure that each team member had continuous access to the latest code developments. We used Trello to track the tasks and progress of each member effectively. However, during the initial phases, we inaccurately assessed our project architecture, misunderstanding Django's framework features, and incorrectly allocated tasks based on frontend and backend roles. Although we identified this issue by the third week of the project and switched our task allocation strategy to assigning tasks based on specific features, this initial mistake inevitably caused tight coupling in our code. As a result, despite consistent effort and contributions from all team members, we experienced difficulties distributing tasks evenly in the project's later stages.

Chapter 5

Design and implementation

5.1 System Architecture

In the development process of our student association management system, we adopted a modular and extensible architecture design to achieve high cohesion and low coupling. The system is built using the Django framework, following the MVT (Model-View-Template) design pattern.

The system architecture adopts Django's MVT pattern, which includes:

- **Model layer:** Defines data structures and database interactions
- **View layer:** Handles business logic and request responses
- **Template layer:** Responsible for interface rendering and display

These three layers work closely together, forming a complete web application process while maintaining the relative independence of each layer.

The system architecture is primarily divided into the following subsystems:

1. **User System:** Responsible for user authentication, registration, and personal profile management
2. **Club System:** Manages student club creation, member management, and activities
3. **Event System:** Handles event creation, registration, and management
4. **Notification System:** Provides real-time notification functionality to users
5. **News System:** Publishes and manages news for each club's activities
6. **Forum System:** Provides a platform for student communication and can be categorized by different topics
7. **Admin System:** Provides backend management functionality for administrators
8. **Message System:** Private messaging functionality between users

Each subsystem is designed as a separate Django application. This modular design allows us to:

- Implement clear separation of functionality

- Simplify team collaborative development
- Improve code maintainability
- Facilitate future functional expansion

5.2 Technology Stack Selection

In our technology selection process, we fully considered the project requirements, development efficiency, and team capabilities, ultimately deciding on the following technology stack:

5.2.1 Backend Technologies

- **Django Framework:** As the main web application framework, providing ORM, authentication, and routing
- **Django REST Framework:** Used to build RESTful API interfaces
- **Django Summernote:** Integrating rich text editing functionality
- **SQLite:** Lightweight database used in the development environment
- **CORS Middleware:** Enabling cross-origin resource sharing support

5.3 Alternative Design Options and Justifications

During the planning phase, we carefully evaluated the following alternative approaches and their pros and cons:

5.3.1 Full CMS Solutions vs. Custom-coded Implementation

We considered using a full CMS solution (like WordPress) to handle rich text posts. However, the assignment requires a custom-coded implementation of core functionality. Instead of relying on a pre-built CMS, we chose Django Summernote as our rich text editor (RTE), which allows us to store media files and properly render rich text content in HTML blocks, while still meeting the requirement of writing our own implementation.

5.3.2 Other Web Frameworks

While some frameworks offer more flexibility or a lighter footprint, Django's built-in features (ORM, authentication, etc.) resulted in more efficient development given our project's timeline and the team's familiarity with Django.

5.3.3 Monolithic vs. Microservices Architecture

Microservices can provide higher scalability for large-scale projects, but our current scope is better suited to a monolithic architecture. We opted not to introduce microservices in order to keep the system straightforward and manage complexity.

5.3.4 Frontend Technologies

- **Bootstrap:** Provides responsive UI components
- **JavaScript/jQuery:** Implements client-side interaction functionality
- **React:** Some modules (such as personal dashboard) are built using React

5.3.5 Storage Solutions

We designed a flexible storage solution for the project that uses local storage in the development environment and can be configured to use cloud storage (such as Amazon S3) in the production environment. This design simplifies the workflow during the development phase while providing scalability for production deployment.

5.4 Data Model Design

The system's data model design follows database normalization principles, ensuring data consistency and integrity. Key model designs include:

5.4.1 User Model

We extended Django's default user model to meet the specific requirements of the student association management system.

5.4.2 Event Model

The event model adopts a many-to-many relationship design, supporting flexible user participation mechanisms.

5.4.3 RSVP Model

To track user participation status for events, we designed a dedicated RSVP model. This design ensures that each user can have only one RSVP record for each event, while also recording the timestamp of status changes.

5.5 Security Mechanism Implementation

Security is an important consideration in our system design. We implemented multi-layered security measures:

5.5.1 Authentication and Authorization

Using Django's authentication system combined with custom middleware. At the view level, we used various mixins to ensure access control.

5.5.2 CSRF Protection

To prevent cross-site request forgery attacks, we strengthened the CSRF protection mechanism and configured trusted origins for both development and production environments.

5.5.3 Password Policy

We adopted Django's recommended set of password validators to ensure users create strong passwords, including similarity validators, minimum length requirements, common password checks, and numeric password validation.

5.6 User Interface Design

5.6.1 Design Style and Principles

We designed a modern, unified, and consistent UI style for the system, adhering to the following design principles:

- **User-Centered Design:** All interface elements are designed to enhance user experience.
- **Consistency:** Ensure visual, interactive, and brand consistency throughout the system, reflecting the student association's unique brand colors and style.
- **Simplicity:** Keep interface elements simple to reduce cognitive load and highlight key functions.
- **Visual Hierarchy:** Establish clear visual hierarchy through color, size, and spacing.
- **Accessibility:** Ensure sufficient contrast and proper element dimensions for accessibility.

5.6.2 Visual Design System

We developed a unified visual design system that includes the following elements:

- **Color Scheme:** Mainly blue (#0071e3) as the primary color, complemented by white, gray, and accent colors to ensure a cohesive look.
- **Typography:** Hierarchical font sizes and weights to improve readability and content structure.
- **Rounded Corners:** Cards, buttons, and input fields feature 12px rounded corners to create a modern and friendly look.
- **Shadow Effects:** Subtle shadow effects provide depth and differentiate interface layers.
- **Icon System:** Utilizing Bootstrap icons and custom icons to ensure a unified icon language.

5.6.3 Component Design

We designed a series of reusable interface components:

- **Card Components:** Primary containers for displaying content with subtle shadow and hover effects to enhance interactivity.
- **Button System:** Hierarchical design of primary, secondary, and text buttons based on their functionality.
- **Form Elements:** Optimized input fields, selectors, and checkboxes to enhance the form filling experience.
- **Navigation Elements:** Carefully designed navigation bars and side menus to provide clear website navigation.
- **Notification Components:** Prominent yet non-intrusive notification design for system feedback.

5.6.4 Responsive Design Implementation

In addition to basic responsive layouts, we paid special attention to:

- **Mobile-First Approach:** Designing for the smallest screen first, then progressively enhancing for larger screens.
- **Flexible Layouts:** Using Flexbox and CSS Grid to achieve dynamic element alignment.
- **Conditional Display:** Showing or hiding specific content based on screen size.
- **Touch Optimization:** Optimizing interactive element dimensions and spacing for touch devices.
- **Performance Considerations:** Optimizing image resources and reducing complex animations to ensure smooth performance on mobile devices.

5.6.5 Interaction Design Patterns

To enhance user experience, we implemented various interaction design patterns:

- **Immediate Feedback:** Provide visual and state feedback after user actions to enhance confidence.
- **Progressive Disclosure:** Present complex information through collapsible panels and tabs to avoid information overload.
- **Microinteractions:** Add subtle animations and transitions to enrich interface liveliness and responsiveness.
- **State Transitions:** Clearly display system state changes, such as button loading states and form validation states.
- **Guided Forms:** Use multi-step form design to lower filling difficulty for complex forms.

5.7 Performance Optimization

To improve system performance and user experience, we implemented multiple optimization measures:

5.7.1 Static Resource Handling

We configured mechanisms for collecting and serving static files efficiently.

5.7.2 Caching Strategy

We implemented caching mechanisms in key selector components to reduce DOM operations and improve interface responsiveness.

5.7.3 Lazy Loading

For complex components and interface elements, we implemented delayed initialization mechanisms to optimize page load performance.

5.8 Extensibility Design

Considering future expansion needs of the system, we particularly focused on the following aspects in our architecture:

5.8.1 Modular Plugin System

Through Django's application mechanism, we implemented a modular plugin system that allows for easy addition of new functionality.

5.8.2 API Interface Design

We built extensible API interfaces using Django REST Framework, facilitating future integration with other systems.

5.8.3 Internationalization Support

Our system design considered internationalization support, making it easier to add multilingual functionality in the future.

5.9 Challenges and Solutions

During the project development process, we encountered several technical challenges and successfully overcame them:

5.9.1 Event Management System

Implementing complex event filtering and categorization functionality to allow users to easily find events by date and category.

5.9.2 Dynamic Form Handling

Using JavaScript to implement dynamic form addition and validation for a more interactive user experience.

5.9.3 Notification System Integration

Seamlessly integrating the notification system with other modules through context processors to provide real-time updates to users.

5.10 Testing Strategy

To ensure system quality, we adopted a comprehensive testing strategy:

1. **Unit Testing:** Targeting core functional modules
2. **Integration Testing:** Verifying interactions between subsystems
3. **User Interface Testing:** Ensuring UI responsiveness and usability
4. **Security Testing:** Checking for potential security vulnerabilities

5.11 Conclusion

Through careful architecture design, technology selection, and implementation strategies, we successfully built a feature-rich, user-friendly, and extensible student association management system. The system's modular design ensures code maintainability, responsive interface design provides a good user experience, and the implementation of security mechanisms safeguards the security of the system and user data. Future extensibility has also been fully considered in the architecture, laying a solid foundation for the system's continuous development.

Chapter 6

Testing

6.1 Approach and tools

In this project, we primarily relied on the Django testing framework for automated testing, which included the following types of tests:

- **Unit Testing:** Testing core functional modules to ensure the correctness of each module.
- **Integration Testing:** Verifying interactions between apps to ensure that various modules work together properly.
- **User Interface Testing:** Ensuring the responsiveness and usability of the user interface (tests focused on template rendering).
- **Security Testing:** Checking for potential security vulnerabilities to ensure system security (tests focused on mixins and other security restriction tools).

6.2 Quality assurance processes

To ensure the quality of our tests, we used test coverage tools for coverage detection. Currently, the test coverage for the entire project reaches 93%, with all tests passing successfully, ensuring the stable operation of the main functionalities.

The test coverage report can be viewed in `./htmlcov/index.html`

6.3 Evaluation of testing

In our testing evaluation, we identified the following strengths and weaknesses:

- **Strengths:**
 - Unit tests and integration tests covered most of the core functionalities, ensuring their correctness.
 - The main user interface templates were tested, ensuring usability.
- **Weaknesses:**

- Some complex functionalities had insufficient test coverage and need further improvement. For example:
 - * Widget functionality in Club Hub
 - * Message chat functionality between users in the Message system
- The tests that could not be automated were compensated through Manual tests, please see Appendix A - Manual tests

Appendix A

Manual tests

#	Test script	Expected outcome	Outcome/date
1	Test the messaging feature between users - Two members log into the deployed version of the website on two different computers with two different accounts and send two messages to each other's accounts.	Both members should receive the two messages sent by the other.	All four messages were successfully sent and received, March 25, 2025.
2	Test the background image feature in Club Hub - One member logs into the deployed version of the website and changes the background image in the AI Club's Hub. Another member logs into a different account to check if the background image is updated correctly.	The background image should be uploaded correctly and displayed properly when logged in with other accounts.	The background image was uploaded correctly and displayed properly, March 26, 2025.
3	Test the widget feature in Club Hub - One member logs into the deployed version of the website and adds a clock widget, an image widget, and a notice widget in the AI Club's Club Hub. Another member logs into a different account to check if the widgets render correctly.	All three widgets should be uploaded and rendered correctly.	All three widgets were uploaded and rendered correctly, March 26, 2025.